might change the object's methods. Good API design and an awareness of this potential problem go a long way toward mitigating the issue.

You could easily have made this test far more brittle if you'd started to worry about tasks being part of projects:

```
it "don't do this" do
  fake_action = double(create: true,
    project: double(name: "Fred", tasks: [double(title: "Start", size: 2)]))
    # and so on …
```

In this snippet, fake_action isn't merely concerned with reporting success; it also wants to stub the project and have the project stub the array of tasks. This is where test doubles become a pain. The preceding snippet is hard to set up, it's hard to read, and it's brittle against changes to object internals. If you find yourself needing to write nested mocks like this, try to restructure your code to reduce dependencies.

Prescription 21

A stubbed method that returns a stub is usually okay. A stubbed method that returns a stub that itself contains a stub probably means your code is too dependent on the internals of other objects.

More Expectation Annotations

RSpec allows a number of different annotations to the expectation part of declaring a test double. You can specify more complex return values or a variety of arguments to the stubbed method.

Stubbing Return Values

A couple of advanced usages of returns might be valuable now and again. If you've multiple return values specified, the stubbed method returns one at a time:

mocks/01/spec/models/task_spec.rb

```
it "stubs with multiple return values" do
  task = Task.new
  allow(task).to receive(:size).and_return(1, 2)
  assert_equal(1, task.size)
  assert_equal(2, task.size)
  assert_equal(2, task.size)
end
```

The return values of the stubbed method walk through the values passed to and_return. Note that the values don't cycle; the last value is repeated over and over again.